Name : Shiladitya Mandal

Enrollment number: 12024052017011

Branch: CSE(IOTCSBT)

Roll : 40

OOPS Lab Project

Project Report: 2D Turn-Based RPG in Java

1. Executive Summary

This project is a custom-built, 2D turn-based role-playing game (RPG) developed entirely in Java. The application leverages core Object-Oriented Programming (OOP) principles to construct a modular and scalable game environment. Key features include a dynamic enemy spawning system, comprehensive inventory and stat management utilizing the Java Collections Framework, and a fully interactive Graphical User Interface (GUI) featuring custom-made art assets.

2. Core Architecture and OOP Design

The game's foundation is built upon robust OOP methodologies to ensure code reusability and maintainability:

- **Encapsulation:** Every entity, weapon, potion, and piece of equipment is designed as an independent object. Each object encapsulates its own distinct set of stats (e.g., health, damage, defense), preventing unintended interference from other game components.
- **Inheritance and Polymorphism:** The architecture supports a hierarchical structure where various game elements (Enemies, NPCs, Items) can extend base

classes and implement interfaces, allowing for a standardized interaction model during gameplay loops and combat sequences.

3. Data Structures and State Management

Efficient data management is critical for game performance, particularly as the number of on-screen elements scales:

- **Entity Management via HashMap:** The Java Collections Framework is heavily utilized, specifically HashMap, to store and manage the myriad of objects and entities. This allows for rapid, O(1) average time complexity retrieval of specific enemies, equipment, or items during active gameplay, ensuring no lag during combat calculations or inventory access.
- **Dynamic Spawning System:** The game features a randomized spawning mechanism. This function dynamically pulls from the available pool of entity objects to generate unique combat encounters, ensuring high replayability and unpredictable gameplay scaling.

4. Graphical User Interface (GUI) and Asset Integration

The visual and interactive layer of the game is driven by native Java UI libraries:

- **Event-Driven Interaction:** The GUI is constructed using JFrame, forming the main application window. User input is strictly handled through interactive buttons mapped to specific actions (e.g., Attack, Defend, Use Item).
- **Action Listeners:** ActionListener interfaces capture these button clicks, translating player input into game logic, calculating turn outcomes, and triggering the next phase of the combat loop.
- **Custom Visual Assets:** Standard UI elements are replaced or overlaid with custom, self-made art. The visual state dynamically updates to display the correct entity images, current equipment, and environmental assets based on the active turn.

5. Future Enhancements

While the current build is highly functional, the architecture supports several avenues for advanced expansion:

- **Advanced Data Handling:** Implementing HashSet collections to track unique discovered items or defeated enemy types, and utilizing Java Streams to allow players to sort and filter their large inventories efficiently.
- **Concurrency:** Integrating multithreading and synchronization to handle background tasks—such as

ambient audio tracks, complex enemy AI calculations, or fluid idle animations—without interrupting the main GUI event dispatch thread.

- **Exception Handling:** Expanding custom wrapper classes and robust exception handling to manage edge cases, such as file-loading errors for custom art assets or corrupted save states.